

Recorrido de árboles o grafos.

algoritmos para recorrerlos solamente una vez:

1) **a lo ancho (BFS)**: Se comienza por el nivel 0 (la raíz) y se visita cada vértice en un nivel antes de pasar al siguiente nivel.

$O(m+n)$

devuelve la distancia de un nodo a todos los otros nodos.

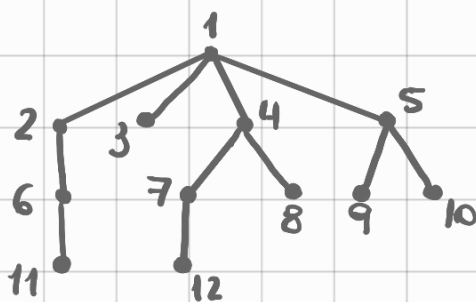
2) **en profundidad (DFS)**: Se comienza por la raíz y se explora cada rama lo más profundo posible antes de retroceder.

El recorrido en DFS no es único.

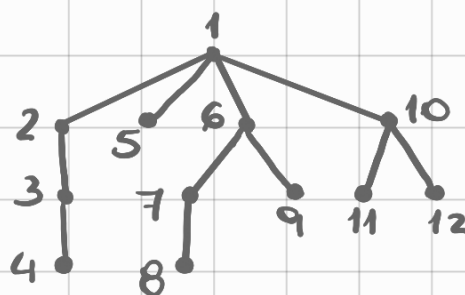
$O(m+n)$.

nos devuelve un árbol.

BFS:



DFS:



- backedges → las aristas por las que no pasé.

Estos dos algoritmos sirven para:

- encontrar todas las comp. conexas de un grafo.
- los puntos de corte de un grafo conexo.
- determinar si un grafo tiene ciclos.
- problema de flujo máximo.
- camino mínimo de un grafo no pesado.
- encontrar vértices que están a distancia menor que k .

Función.

los valores pred definen un AG de G .

recorrer (G)

Salida: $pred[i] = \text{Padre de } v_i$ - $orden[i] = n^\circ \text{ asignado a } v_i$

$next = 1$

$r = \text{elegir un vértice como raíz}$

$pred[r] = 0$

#marcar vértice r

$orden[r] = next$

$lista = \{r\}$

while $lista \neq \emptyset$ do

elegir un nodo i de lista

if $\exists \text{ un arco } (i, j) / j \notin lista$ then

$pred[j] = i$

#marcar vértice j

$next = next + 1$

$orden[j] = next$

$lista = lista \cup \{j\}$

else

$lista = lista \setminus \{i\}$

return $pred$ y $orden$

acá se marca la diferencia entre BFS y DFS.

BFS $\rightarrow$ lista implementada como cola.

DFS $\rightarrow$ lista implementada como pila.

* ¿Cómo podemos ver que un grafo es conexo?

Hacemos BFS desde un nodo. Guardamos en un vector las distancias a todos los nodos. Recorremos el vector distancias.

Si un valor no está ahí, significa que no hay camino a ese nodo.

→ BFS o DFS desde un nodo veo si todos los nodos fueron visitados (usando el vector visitados).

* Caminos con min. cant. de cuerdas entre R y F.

BFS en R. Nos da la dist. a otro nodo (en particular hasta F).

rechenos que corta la cant. min. de caminos.

¿cuántos árboles BFS se pueden conseguir?



descompongo el camino en la unión de varios subcaminos

$$d(R, F) = d(R, i) + d(i, j) + d(j, F)$$

si asumo que $\exists$ camino más chico que (x.ej.) i, j

$\Rightarrow$ llego a un abs pues mi camino no sería el mínimo.

$\text{dist}(F, R) = k$. desde F me fijo los que están a dist $k-1$ y que sean vecinos de F. Sino, obtendré un camino más largo.

caminos Hasta F = $\sum$ # caminos Hasta los Hijos $k-1$

$$\# \text{caminos Hasta } (v) = \begin{cases} 1 & ; R = v \\ \sum_{w \in N_v} \# \text{caminos Hasta } (w) \end{cases}$$

Función recursiva

$$w = \{w \in N_v : d(w) + 1 = d(v)\}$$

recibe sólo un vértice.

Se inicializa con caminos Hasta (F)

$O(n+m) \rightarrow$ BFS

prop. G es bipartito $\Leftrightarrow$ toda comp. conexa lo es

demo.

$\Rightarrow$) Sean $\{G_i\}$ con $G_i = (V_i, E_i)$ las comp. conexas.

Sabemos que $\exists w, z$ bipartición de G .

$\forall v \in V_i, v \in w, v \in z$ es bipartición de G_i .

$$\bullet (v_i \cap w) \cup (v_i \cap z) = v_i \cap (w \cup z) = v_i \cap V = v_i$$

$$\bullet (v_i \cap w) \cap (v_i \cap z) = v_i \cap (w \cap z) = v_i \cap \emptyset = \emptyset$$

Sean $w_1, w_2 \in v_i \cap w \subset w \Rightarrow (w_1, w_2) \notin E \Rightarrow (w_1, w_2) \notin E_i$

$\Leftarrow$) Sean w_i, z_i biparticiones de G_i .

$\forall v \in V, \bigcup w_i, \bigcup z_i$ es bipartición de G

$$\bullet \bigcup w_i \cup \bigcup z_i = \bigcup (w_i \cup z_i) = \bigcup V_i = V$$

$$\bullet \bigcup w_i \cap \bigcup z_i = \bigcup (w_i \cap z_i) = \bigcup \emptyset = \emptyset$$

Caso 1: $w_1, w_2 \in w_i \Rightarrow (w_1, w_2) \notin E_i \Rightarrow (w_1, w_2) \notin E$

Caso 2: $w_1 \in w_i, w_2 \in w_j$ con $i \neq j \Rightarrow (w_1, w_2) \notin E$

Análogo para z .

prop. Si w, z es una bipartición. Sea G conexo.

Sea $w_1 \in w \Rightarrow \forall w \in w, d(w_1, w)$ es par y $\forall z \in z, d(w_1, z)$ es impar.

$d(w, v)$ es par $\Leftrightarrow v \in w$.

corolario. Si tengo que $w, z \subset V / w \cup z = V, w \cap z = \emptyset /$

dado $v, w \in w, z \in z, d(v, w)$ es impar o $d(v, z)$ es par
 $\Rightarrow$ No es bipartito.

prop. Si G es bipartito, hay un solo modo de particionarla.

prop. G bipartito $\Leftrightarrow$ no tiene ciclos impares.

Ejercicio 1). Dar algoritmo lineal basado en BFS para ver si un grafo es bipartito.

Si G conexo:

BFS(G, v)

for $(w, z) \in E$

if $d.w + d.z$ es par

return false

return true

• $w = \{w : d(v, w) \text{ es par}\}$

• $z = \{z : d(v, z) \text{ es impar}\}$

$d.w = d(v, w)$

$d.z = d(v, z)$

lema. $e = (u, v)$ es puente $\Leftrightarrow$ no hay camino entre u y v en $G \setminus e$.

demo. 2 casos

$\Rightarrow$) e puente. Por absurdo. Asumo P camino entre u y v .

① G conexo. $G \setminus e$ desconexo $\Rightarrow \exists x, y \in V(G) /$ no hay camino entre x y y en $G \setminus e$

$\langle x \rightarrow u \rangle \not\subseteq (u, v)$

$\langle y \rightarrow v \rangle \dots$

• $\langle y \rightarrow v \rangle \ni (u, v)$ $\langle y \rightarrow u \rangle \not\subseteq (u, v)$

$\langle y \rightarrow u \rangle + + \langle x \rightarrow u \rangle^{-1}$ es un camino de $G \setminus e$. Abs!
lo doy vuelta

• $\langle y \rightarrow v \rangle \not\subseteq (u, v)$

$\langle y \rightarrow v \rangle + + P^{-1} + + \langle x \rightarrow u \rangle^{-1}$ es un camino entre x y y en $G \setminus e$

Abs!

⑪ G desconexo. u y v pertenecen a una comp. conexa $\mathcal{C}$.
 e puente en $G \Leftrightarrow e$ puente en $G(\mathcal{C})$

$\Leftarrow$). $\nexists$ camino entre u y v en $G \setminus e$. $\forall v \in \mathcal{C}$ es puente.
Por abs. ASUMO que e no es puente. $\Rightarrow$ la comp. conexa $\mathcal{C}$
la que pertenece e en G es conexa en $G \setminus e$. $\Rightarrow$ hay
camino entre u y v . Abs!

lema. Una arista es puente $\Leftrightarrow$ no pertenece a ningún ciclo
demo.

$\Rightarrow$). $e = (u, v)$ puente. $\forall v \in \mathcal{C}$ $\nexists$ ciclo que lo contenga.

ASUMO que $\exists$. $C = (v_1, \dots, u, v, \dots, v_1)$

$C[v_1, v_1] \cup C[v_1, \dots, u] = P$. Abs! por lema anterior.

$\Leftarrow$). $e \notin$ a ningún ciclo. ASUMO que e no es puente.

Por lema anterior, $\exists \langle u \rightarrow v \rangle$ en $G \setminus e \Rightarrow \langle u \rightarrow v \rangle \cup (u, v)$
es un ciclo en G .

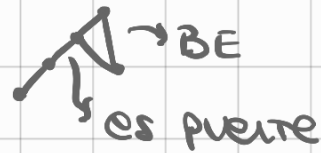
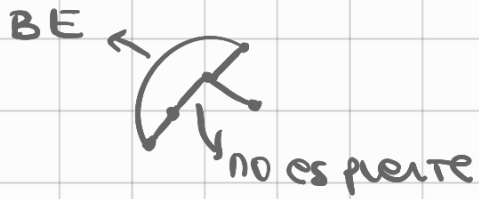
lema. Toda arista de un grafo conexo es o una tree-edge
o una backward-edge después de correr DFS.

lema. Una backedge no es puente. Si hay BE, hay ciclo.

lema. Una tree-edge pertenece a un ciclo $\Leftrightarrow$ existe una
backedge entre un descendiente de u y un ancestro de v .

lema. Sea G conexo toda arista de G en DFS es una TE
o una BE

Lema. una TE es puente $\Leftrightarrow$ no hay BE que los cubra



Ejercicio 2). Dado un grafo G , encontrar los puentes de G en $O(n+m)$.

1. DFS con padres, tiempos
2. identifico tree-edges
3. detecto cuáles están "cubiertas"

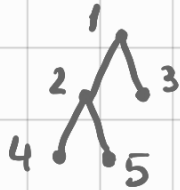
$$\text{minNivelBackedge}(v) = \min \{ \text{nivel}(v), \text{minNivelAlcanzado por una Backedge de } v, \min \{ \text{minNivelBackedge}(w) \} \}$$

$\downarrow$
 hijo de v .

$$\sum_v O(d(v)+1) = O(2m+n) + O(m+n)$$

Tipos de aristas en DFS

Tree-edges: las usadas para descubrir nuevos vértices. Es una arista que forma parte de la estructura del árbol y conecta dos vértices dentro de ese árbol.



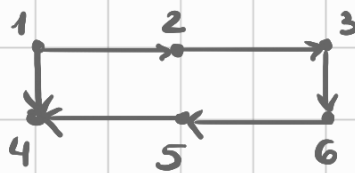
$$TE = (1, 2), (1, 3), (2, 4), (2, 5)$$

backward - edges: las que van de un vértice a un ancestro. Se encuentran en grafos dirigidos. Conecta a un nodo con uno de sus ancestros.



$$BE = \underbrace{(1, 4)}_{\substack{4 \text{ ancestro} \\ \text{de } 1}}, \underbrace{(3, 6)}_{\substack{6 \text{ ancestro} \\ \text{de } 3}}$$

forward - edges: las que van de un vértice a un descendiente que no sea un hijo. No es una TE directa.



$$FE = (1, 2), (2, 3), (6, 5), (5, 4)$$

cross - edges: sólo en grafos dirigidos las que van de un vértice a otro vértice ya visitado que no es ni descendiente ni ancestro. Conecta nodos que están en diferentes ramas del árbol DFS.



$$CE = (4, 1), (3, 6)$$